

Teaching Verification-Centered Software Engineering

with Coding Agents

From candidate patches to evidence-backed claims and human judgment



Chenxi Luo · Moon-Kuen Mak · Wei Xiang · Tiejian Luo*
NeurIPS 2026 Education Track · * Corresponding author: tjluo@ucas.ac.cn

Instructor Notes

This opening slide frames the whole workshop: AI may produce candidate patches quickly, but software engineering begins only when we ask what evidence supports a claim and who is accountable for the decision.

Use the analogy of a lab result: a number on a screen is not a medical diagnosis until the method, scope, uncertainty, and clinician judgment are clear.

In practice, learners should remember the four words on the slide as a workflow—Evidence, Claim, Judgment, Decision—rather than treating code generation as the final step.

What this module is—and is not

Avoid the common confusion with a self-improving agent course.

Self-improving agent course

- Improve the agent
- Train/RL, search, reward models
- Planner or agent architecture
- Benchmark/task performance
- Original agent research

This module

- Improve the engineering decision
- Validate intent and bound the agent
- Strengthen independent evidence
- Calibrated claim + justified approval
- Reusable 150-minute SE workshop

The agent may improve its candidate; the learner must improve the evidence and judgment.

Instructor Notes

This slide prevents a common misunderstanding: the module is not about making the agent smarter, but about making the engineer's decision more trustworthy.

A useful analogy is coaching: a self-improving agent course trains the athlete, while this workshop trains the referee and safety officer who decide whether the play should count.

When doing the labs, learners should focus less on prompt tricks and more on whether the intent was validated, the agent was bounded, the evidence was independent, and the approval was justified.

Longer agent tasks, larger review burden

A final green signal hides a longer chain of assumptions and actions.



More search can improve a candidate.

More autonomy can expand the impact of a mistake.

Neither defines the evidence required to accept a software change.

Educational problem: trustworthy engineering judgment around agent-produced change.

Instructor Notes

This slide explains why the topic matters now: as agents touch more files, tools, and steps, a single final green check hides a much longer chain of assumptions.

Think of it like approving a long supply chain from only the final delivery receipt; the package arrived, but that does not prove every upstream action was safe or correct.

In practice, every lab asks learners to reconstruct the chain behind the result before deciding whether a software change is acceptable.

Learning outcomes

By the end, learners can use evidence, not prompt fluency, as the unit of progress.

Classify

evidence, revision
guidance, model
opinion, approval

Execute

evidence-to-approval
lifecycle under a
bounded ACI

Expose

weak verifier with
boundaries and
mutations

Write

property-scoped
claim with residual
uncertainty

Transfer

architecture/security
judgment to
authorization

Remember: the assessment target is calibrated judgment, not lines generated.

Instructor Notes

This slide translates the workshop into concrete learner capabilities, from classifying signals to transferring judgment into a new domain.

The analogy is pilot training: success is not measured by how many buttons the pilot presses, but by whether the pilot can interpret instruments, handle abnormal signals, and make a safe decision.

During the labs, learners should use these outcomes as a checklist for their own work products: classification, lifecycle execution, falsification, scoped claim, and transfer.

The emerging technical concept: improve the engineer

Coding agents are interactive, but this module trains the learner's evidence loop.



Our learner—not the agent—is the primary object of improvement.

Success = reproducible evidence + correctly scoped claim + justified decision

Instructor Notes

This slide introduces the central shift: coding agents are interactive systems, but the primary learner improvement is in the human evidence loop.

A coding agent is like an advanced driving assistant: it can steer, accelerate, and react, but the licensed driver is still responsible for understanding road conditions and deciding when control is safe.

Learners should therefore treat every agent action as something to inspect, reproduce, and verify, not as an automatically acceptable result.

The full evidence-to-approval lifecycle

Separate specification, generation, evidence, revision, claim, and judgment.



Revision guidance R loops back only on counterexample

CLEVER

Intent ↔ Spec

SWE-agent

Agent ↔ Environment

EvalPlus

Candidate ↔ Verifier

ReLoc

Feedback → Revision

This module

Claim → Judgment

Instructor Notes

This slide gives the full lifecycle from intent to approval, so students can see where each research paper and each lab activity fits.

The analogy is a clinical pathway: symptoms become a diagnosis hypothesis, tests produce evidence, treatment is revised, and a physician makes an accountable decision rather than blindly following one test result.

In the labs, learners should point to the exact lifecycle stage they are working on—specification, candidate, verifier, revision, claim, or judgment—so they do not collapse different responsibilities.

A compact technical model

A lightweight notation helps students name each responsibility.

$$I \rightarrow S \rightarrow (\text{Agent} + \text{ACI}, X) \rightarrow C \rightarrow V \rightarrow \{E, F\} \rightarrow R \rightarrow C' / K \rightarrow J$$
I

stakeholder intent before validation

S

intended behavior or specification

X

repository and execution environment

C

generated candidate

V

executable verifier

E

supporting evidence

F

falsifying counterexample

R

guidance about what to revise next

K

property-scoped claim

J

accountable engineering judgment

Instructor Notes

This slide provides a compact notation that lets learners name each object in the feedback loop without getting lost in implementation details.

Treat the notation like a circuit diagram: it does not show every wire in the machine, but it identifies the components and the direction of responsibility.

When completing lab writeups, learners should use these symbols to state what changed, what was tested, what evidence was produced, and what claim is justified.

Three signals that must not be collapsed

Students must learn what each signal permits—and what it does not.

Correctness evidence

What property is supported?
tests, mutation/static analysis, proof,
policy check

Revision guidance

What should be tried next?
reward, self-evaluation, revision
distance, search score

Approval judgment

Should this enter the lifecycle?
architecture, security, compliance,
rollback, owner decision

Model explanation is a fourth category: an opinion or rationale, not independent evidence.

Instructor Notes

This slide teaches one of the most important distinctions in the module: evidence, revision guidance, and approval judgment answer different questions.

An analogy is weather planning: a thermometer is evidence, a forecast suggests what to try next, and the school principal decides whether to cancel an outdoor event.

In practice, learners should label each signal before acting, because using a reward score as approval or a model explanation as evidence is a category error.

Signal-classification checkpoint

Classify first. Decide only after you know what the signal permits.

Item

pytest: 12 passed

reward-model score 0.91

LLM says "looks correct"

static analyzer: no finding

architect rejects global state

accountable reviewer approves

Category

correctness evidence
scoped to named tests/environment

revision guidance

model opinion

correctness evidence
scoped to rules

approval judgment with architecture evidence

approval decision

No single row establishes universal correctness.

Instructor Notes

This checkpoint turns the previous theory into a quick decision exercise: students classify signals before deciding what each signal permits.

It works like sorting documents in court: testimony, physical evidence, expert advice, and a judge's ruling have different roles even if all appear in the same case file.

Learners should practice saying, 'This supports only this scoped property,' or 'This only suggests the next revision,' before they move toward acceptance.

Deliberate scope boundary

Holding the agent fixed makes the evidence and judgment visible.

This workshop does not teach

model training or reinforcement learning
inference-time scaling or repeated sampling
MCTS, planner optimization, agent architecture search
prompt optimization, memory systems, meta-agents

This workshop focuses on

validated intent
bounded agent interface
independent evidence
claim calibration
engineering judgment

Instructor Notes

This slide deliberately narrows the workshop so the learning target stays visible: we are not teaching model training, search algorithms, or agent architecture design.

The analogy is using a microscope: by holding the agent mostly fixed, we can focus sharply on evidence quality and judgment instead of chasing every possible AI technique.

During the workshop, learners should resist optimizing prompts or tools prematurely and instead ask whether the current evidence is sufficient for the decision being made.

What makes a verifier independent?

Independent evidence does not merely repeat the candidate's assumptions.

Agent says "looks correct"



Separately designed oracle

Test duplicates implementation



Boundary/property from validated intent

Same model grades itself



Tool, checker, or accountable reviewer

"All tests passed"



Named tests, environment, logs, limitations

Key move: design the oracle from validated intent—not from the candidate.

Instructor Notes

This slide defines independent verification as evidence that does not merely repeat the candidate's own assumptions.

A practical analogy is measuring a table with a separate ruler rather than asking the carpenter who cut the wood whether it looks straight.

In the labs, learners should design oracles from validated intent and boundary behavior, not by copying the implementation structure or the model's explanation.

Three limits of executable verification

Verification reduces uncertainty; it does not create universal correctness.

Specification risk

S may not represent real intent.

Verifier risk

V may omit a behavior or accept a mutation.

System risk

Passing behavior may violate architecture or security.

A stronger verifier cannot repair the wrong intent.

Instructor Notes

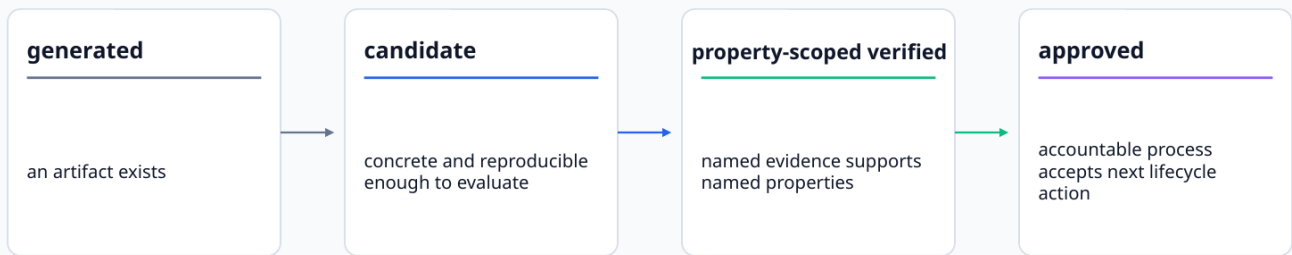
This slide shows why executable verification is powerful but limited: the specification may be wrong, the verifier may be incomplete, and the system design may still be risky.

Think of a building inspection: an electrical inspection can pass while the floor plan still fails the user's needs or the fire escape remains unsafe.

Learners should therefore treat every passing result as uncertainty reduction, then ask what remains unverified before making a claim or approval decision.

Four lifecycle states

Do not jump from generated to merged.



Generated → Candidate → Property-scoped verified → Approved

Instructor Notes

This slide replaces the dangerous shortcut from generated code to merged code with four lifecycle states that require increasing levels of evidence and accountability.

A useful analogy is airport security: having a ticket, checking in, clearing security, and boarding are related but separate states with separate gates.

In practice, learners should label artifacts precisely as generated, candidate, property-scoped verified, or approved, because each label permits different actions.

Four research surfaces—and our fifth layer

Each paper exposes a distinct failure surface; this module integrates them as a teachable workflow.

EvalPlus	weak evidence	augmented tests + mutations
SWE-agent	constrained interaction	ACI actions + concise feedback
CLEVER	specification fidelity	semantic equivalence + proof
ReLoc	revision guidance	candidate evaluation + local search
This module	claim calibration	engineering approval

Our contribution is not a fifth algorithm; it is the educational integration layer.

Instructor Notes

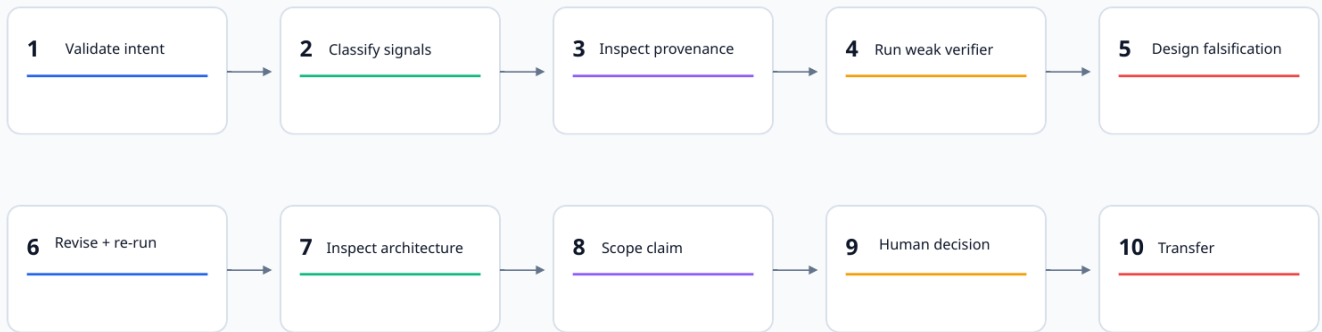
This slide connects the workshop to four frontier research surfaces and explains the module's fifth layer: claim calibration and engineering approval.

The analogy is four diagnostic lenses plus a final clinical judgment: tests, interfaces, specifications, and revision policies reveal different problems, but none alone decides treatment.

Learners should use the research papers as motivation for the workflow, while recognizing that the educational contribution is integrating them into software engineering practice.

Workshop route

A memorable route from intent to transfer.



Instructor Notes

This slide gives learners a memorable route through the workshop, from validating intent to transferring the method into a new domain.

It is like a hiking trail with numbered markers: if learners skip a marker, they may still move forward but lose track of why the next step is justified.

In practice, teams should use this route as their lab checklist and record one artifact at each step, especially evidence, residual uncertainty, and decision rationale.

Lab 1: specification risk comes first

Prompt: "Make discounts reliable near the threshold."

Observed

facts in artifacts

Inferred

hypotheses from learner/AI

Validated

confirmed stakeholder intent

Before code: identify user, scenario, value; ask whether threshold is inclusive; design acceptance examples; record validation.

Instructor Notes

This slide begins Lab 1 by putting specification risk before code, using the discount threshold as a simple but revealing example.

The analogy is writing a recipe: before cooking, you must know whether 'at 100' means included or excluded, otherwise even perfect execution may produce the wrong dish.

Learners should separate observed facts, inferred hypotheses, and validated stakeholder intent before they write or accept any test.

Lab 2: candidate provenance

A prepared candidate makes the result reproducible without an API key.

- frozen baseline
- exact unified diff
- provenance manifest
- learner-visible tests
- state candidate, not verified

```
candidate_id: C-017  
base_commit: 4f91e2a  
diff: patches/C-017.diff  
visible_tests: tests/test_visible.py  
status: candidate_not_verified
```

Provenance turns “the AI changed something” into a reproducible object.

Instructor Notes

This slide makes the candidate patch reproducible by recording its baseline, diff, manifest, tests, and status.

Think of provenance as a chain of custody for software: without it, you cannot later explain exactly what object was evaluated or how it was produced.

In practice, learners should never say 'the AI fixed it' without naming the candidate ID, base commit, diff, tests, and current verification status.

Lab 3: the weak verifier passes

The green check is a question, not an answer.

```
$ pytest tests/test_visible.py -q  
4 passed
```

Question:

What plausible incorrect implementation would still pass?

Do not inspect the instructor suite until you have designed your own falsification cases.

Instructor Notes

This slide creates the first aha moment: the visible verifier passes, but that green check is a prompt for investigation rather than proof of correctness.

The analogy is a smoke detector in only one room: silence tells you something about that room, but not about the whole building.

Learners should immediately ask what plausible incorrect implementation could still pass the current tests, then design falsification cases before seeing the instructor suite.

Facilitator pause: learner falsification attempt

Stop here before revealing the worked counterexample.

- 1 commit at least one plausible mutation
- 2 design tests that would kill it
- 3 execute those tests against Candidate C-017
- 4 record evidence and residual uncertainty

Continue only after artifacts have been collected.

Instructor Notes

This slide is a facilitator pause that protects productive struggle before the worked answer is revealed.

It resembles a science lab where students must write a hypothesis and run their experiment before the teacher shows the expected result.

In practice, require learners to commit a mutation, design a killing test, execute it, and record residual uncertainty so the lesson becomes experiential rather than verbal.

Strengthen the oracle

Design tests from validated intent, not from implementation structure.

equivalence classes

exact boundaries

just-below / above cases

invalid inputs

plausible mutations

properties not copying code

```
assert discount(price=100, threshold=100) == expected
# stronger suite reveals a counterexample at the exact threshold
```

Instructor Notes

This slide teaches how to strengthen an oracle using boundaries, equivalence classes, invalid inputs, mutations, and properties that do not copy the code.

The analogy is stress-testing a bridge at joints, limits, and unusual loads rather than only driving a small car across the middle once.

Learners should turn validated intent into targeted tests, especially exact-threshold and just-below/above cases, then revise the claim based on what those tests actually establish.

Lab 4: same outputs, different systems

Both candidates pass 12 shared functional tests. Now inspect engineering risk.

process-global mutable state

direct environment access

dependency direction

duplicated validation

behavior after configuration changes

test isolation and rollback

Functional correctness is not the same as engineering acceptability.

Instructor Notes

This slide moves beyond functional outputs and asks whether a passing implementation is architecturally acceptable.

Two cars may both arrive at the destination, but one may rely on broken brakes or an unsafe shortcut that should not be accepted in a fleet.

In practice, learners should inspect global state, hidden environment access, dependency direction, duplicated validation, isolation, and rollback consequences before approving a design.

An executable architecture counterexample

Functional examples did not encode determinism under configuration change.

```
Candidate A, default policy: 12.0  
Candidate A, after rate changes: 12.0 <- stale  
cache  
Candidate B, explicit new policy: 24.0
```

Architectural lesson

A passing output can hide stale state, hidden dependencies, or rollback risk.

A verifier can prove a property only if the property is actually checked.

Instructor Notes

This slide gives an executable architecture counterexample: Candidate A passes examples but fails when configuration changes because it hides stale state.

The analogy is a thermometer stuck at yesterday's temperature; it may look reasonable in one case, but it cannot be trusted under changing conditions.

Learners should ask what system property was not encoded in the tests, then add a verifier or decision rule that checks determinism under configuration change.

Bound the agent, not only the prompt

The contract turns “agent freedom” into inspectable interface design.

Objective

validated acceptance criteria

Scope

included / excluded paths

Tools

allowed and forbidden actions

Verification

commands and expected results

Stop

escalation conditions

Approval

actions requiring human approval

Compare unrestricted shell vs bounded contract: differences are interface-design evidence, not model improvement.

Instructor Notes

This slide shows that bounding the agent is an engineering control, not merely a prompt-writing preference.

A task contract is like a construction permit: it defines the job, allowed tools, restricted zones, inspections, stop conditions, and decisions requiring human approval.

In practice, learners should compare unrestricted and bounded contracts to see how interface design changes the evidence available for review.

Lab 5: non-isomorphic transfer

New domain: document authorization after collaborator offboarding.

Principals

users, roles, service accounts

Resources

documents, folders, links

Grants

share links, group access

Revocations

offboarding, expiration

Precedence

deny overrides allow?

Copying the discount boundary test does not demonstrate transfer.

Rebuild executable policy evidence, architecture, privacy, audit, and rollback judgment.

Instructor Notes

This slide tests real transfer by moving from discount logic to document authorization, where the domain objects and risks are different.

The analogy is changing sports: knowing how to serve in tennis does not prove you can pass in soccer, even though both require coordination and rules.

Learners must rebuild principals, resources, grants, revocations, precedence, policy evidence, architecture, privacy, audit, and rollback judgment rather than copying the previous boundary test.

Assessment and pilot plan

Measure what matters in AI-assisted engineering education.

calibrated pre/post concept answers

correct separation of evidence, guidance, opinion, approval

claims linked to executable evidence

hidden insufficiency discovered before release

architecture decision quality

non-isomorphic transfer score

completion time and learner friction

Do not grade prompt count, token count, or lines generated.

Instructor Notes

This slide defines assessment around calibrated engineering judgment instead of surface measures such as prompt count, token count, or lines generated.

It is like grading navigation rather than fuel consumption: the question is whether the team reached the right destination safely, with evidence, not how much activity occurred.

In practice, evaluate whether learners separate signal categories, link claims to executable evidence, discover hidden insufficiency, justify architecture decisions, and transfer the method.

The agent proposes.

Evidence scopes the claim.

Engineering judgment approves or rejects.

When code becomes cheap, teach learners to design the evidence loop.

Resources

data, evidence, reputation

Rubrics

evidence + confidence

Pilot

proposal + reputation data

Instructor Notes

This final slide leaves the audience with the operating mantra: the agent proposes, evidence scopes the claim, and engineering judgment approves or rejects.

The analogy is a courtroom: a witness may make a statement, evidence constrains what can be believed, and a judge makes an accountable decision under defined rules.

After the workshop, learners should apply this mantra whenever code becomes cheap: do not reward generation volume; design the evidence loop that makes a software change trustworthy.